

Git ve GitHub: Ekip Çalışmasının Temelleri

Versiyon kontrolünden bulut iş birliğine — modern yazılım geliştirmenin temel araçlarını öğrenin. Bu sunum, Git ve GitHub'ı sıfırdan kurumsal düzeyde kullanmanız için gereken her şeyi kapsar.

</> VERSİYON KONTROL

☁ BULUT İŞ BİRLİĞİ

👥 TAKIM YÖNETİMİ

Büyük Resim: Git ve GitHub Nedir?

Git Nedir?

*Kod değişikliklerimizi **bilgisayarımızda izleyen**, hatalı işlemlerde geri dönmemizi sağlayan bir **versiyon kontrol sistemidir**. Tıpkı bir oyundaki "Save Point" gibi çalışır — istediğiniz noktaya geri dönebilirsiniz.*

GitHub Nedir?

*Git altyapısını kullanarak projelerimizi **bulutta depoladığımız**, takımımızla paylaştığımız ve üzerinde **birlikte çalıştığımız** bir servistir. Git yerel bir araçken, GitHub bu aracı internet üzerinden herkesin kullanımına açar.*

Git = Yerel Araç

Bilgisayarınızda çalışır, değişiklikleri takip eder

GitHub = Bulut Servisi

Projeleri internette depolar, ekiple paylaşır

Birlikte = Güç

Takım iş birliği ve kod yönetiminin temeli

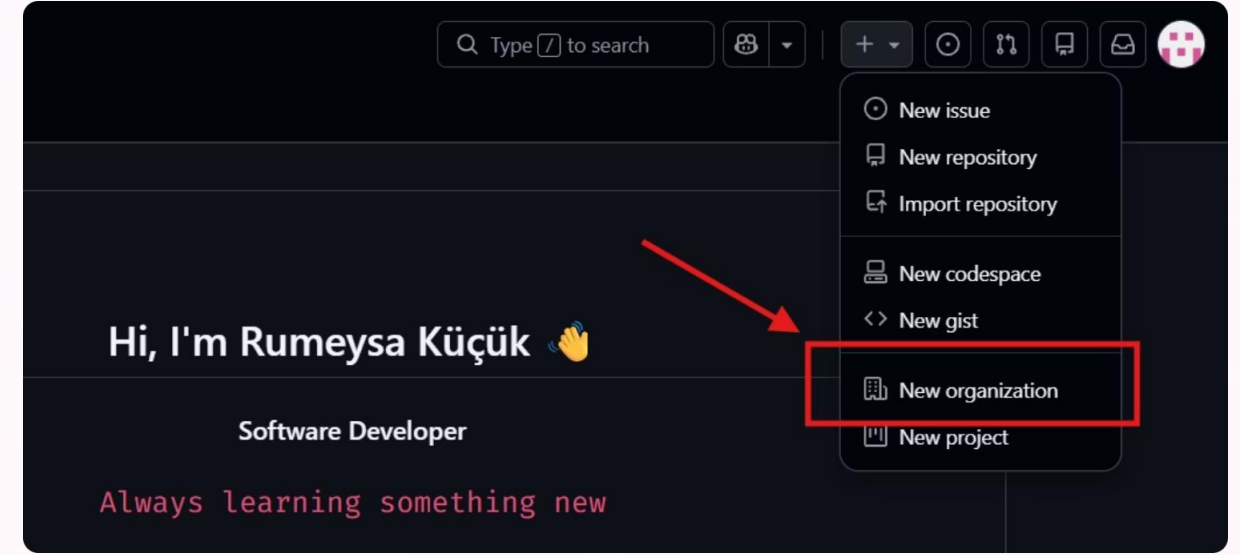


Organizasyon ve Takım Mimarisi

Organizasyon Nedir?

Şirketlerin veya büyük projelerin **birden fazla repository** ve **kullanıcıyı tek çatı altında yönettiği** alandır. GitHub ana sayfasında sağ üstteki + ikonuna tıklayıp **"New organization"** seçeneği ile kurulur.

 Organizasyon, tüm ekibin ortak çalışma alanıdır. Tüm repolar ve takımlar buradan yönetilir.



1

Takım Oluşturma

Kişilere repolara göre topluca yetki vermek için takımlar oluşturulur: **okuma, yazma, admin** seviyeleri mevcuttur.

2

Takımdan Çıkarmak

Üye organizasyonda kalır ancak **sadece o takımın sorumlu olduğu repolardaki yetkilerini** kaybeder.

3

Organizasyondan Atmak

Üyenin sistemdeki projelerle, verilerle ve gruplarla **tüm ilişkisi tamamen silinir.**

Temel Kavramlar: Repository ve Eriřim Türleri

Repository (Repo) Nedir?

Projeye ait tüm dosyaların ve Git tarihçesinin saklandığı dizindir. Her commit, her deęişiklik, her dal — hepsi repo içinde tutulur.

Public / Private Farkı

- **Public Repo:** İnternetteki herkes tarafından görülebilir. Açık kaynak projeler için idealdir.
- **Private Repo:** Sadece izin verdiğiniz takım üyeleri tarafından görülebilir ve deęiřtirilebilir.

Public Repo

Herkes görebilir

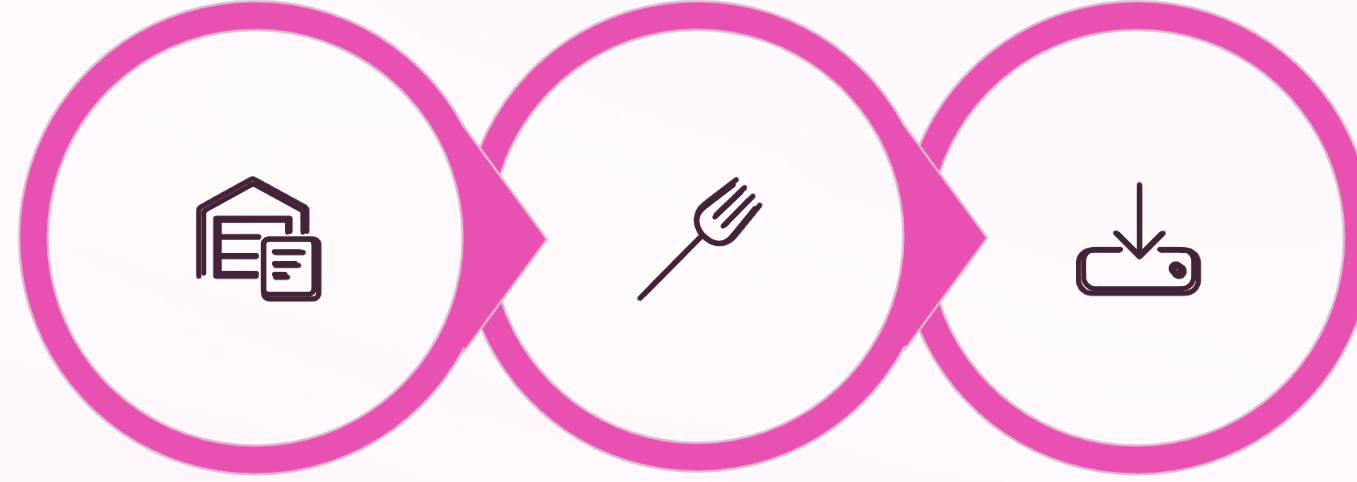
Açık kaynak, portföy projeleri

Private Repo

Sadece yetkili üyeler

Şirket projeleri, gizli kodlar

Temel Kavramlar: Fork ve Clone



Ana Repo

Fork

Clone

Bu üç adım, güvenli ve düzenli bir iş akışının temelidir. Her geliştirici kendi alanında çalışır, ana projeye zarar vermez.



Fork Nedir?

Başkasına ait bir ana repoyu, bulut üzerinde **kendi GitHub hesabınıza** birebir kopyalamaktır. Ana projeden bağımsız, kendi güvenli çalışma alanınızı oluşturur.



Clone Nedir?

Buluttaki bir repoyu **yerel bilgisayarınıza** indirme işlemidir. `git clone [url]` komutuyla yapılır ve kod üzerinde çevrimdışı çalışmanızı sağlar.



Neden Önce Fork?

Herkesin ana projeye **doğrudan kod yazmasını** (yetki karmaşasını) engellemek için. Herkes kendi güvenli alanında çalışıp sonradan izinle kodunu ana repoya gönderir.

Paralel Evrenler: Branch ve Commit

Branch (Dal) Nedir?

Ana projeye zarar vermeden, **yeni özellikler denemek için ayrılan paralel yoldur**. Tıpkı bir ağacın gövdesinden çıkan dallar gibi — her dal bağımsız gelişir.



Neden main dalında çalışmak yasaktır? *main dalı her zaman müşteriye/yayına hazır, stabil çalışan kodun olduğu yerdir. Geliştirmeler her zaman yan dallarda (**feature branch**) yapılmalıdır.*

1

main Branch

Stabil, yayına hazır kod

2

feature Branch

Yeni özellik geliştirme

3

git add

Değişiklikleri sahneye al

4

git commit

Değişiklikleri mühürle

Commit (Mühürlemek)

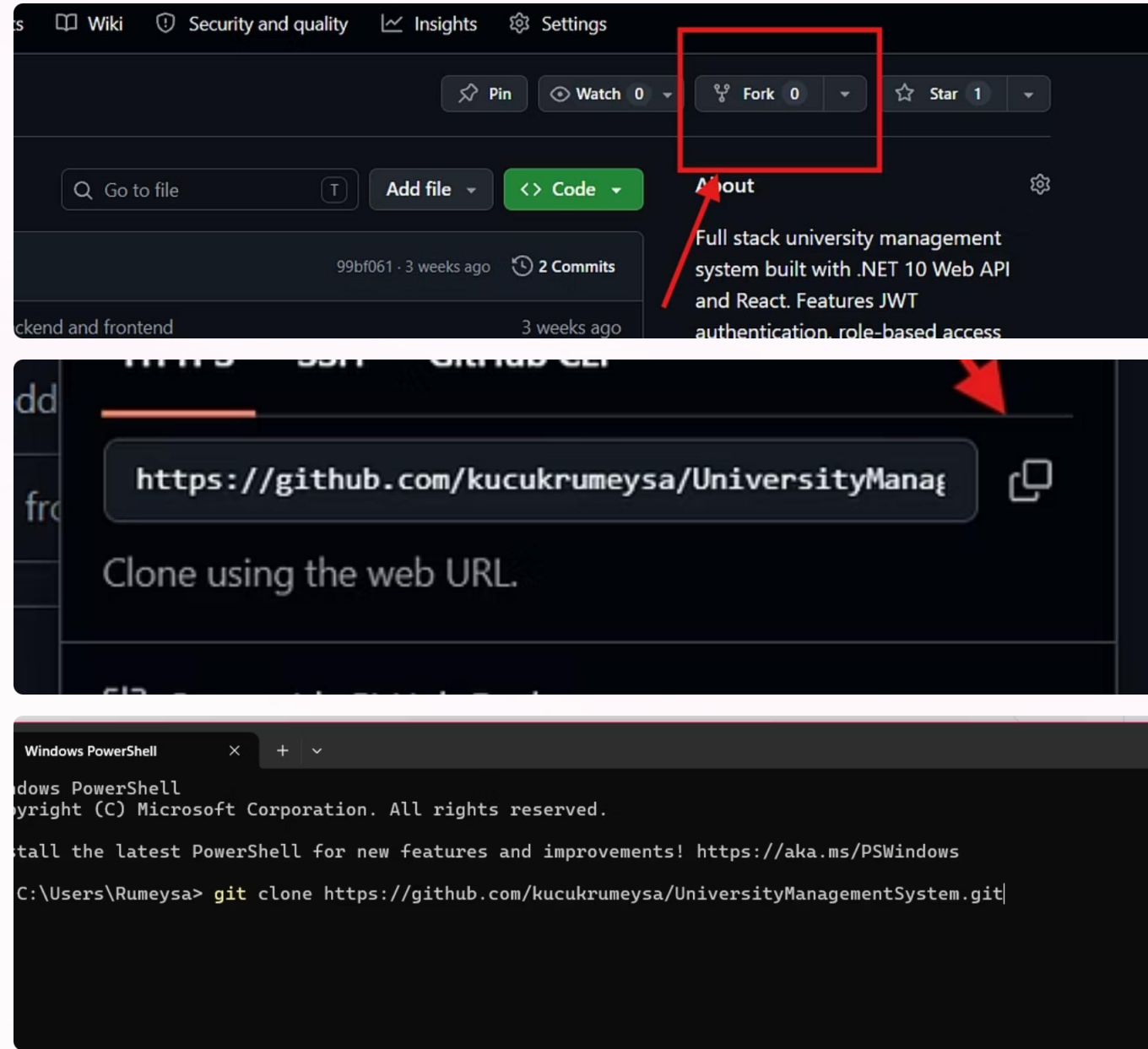
Yaptığınız değişiklikleri bir açıklamayla paketleyip **Git tarihçesine kalıcı olarak kaydetmektir**.

İyi Bir Commit Nasıl Olmalı?

- Ne yaptığınızı net özetlemelidir
- `feat: kullanıcı giriş ekranı eklendi`
- `fix: şifre sıfırlama hatası düzeltildi`



Operasyonel Döngü I: Kodlamaya Başlamadan Önce



Kodlamaya Başlamadan Önce 3 Adım

1 Fork Etme

GitHub'da projenin sağ üst köşesindeki "Fork" butonuna tıklayarak projeyi kendi hesabımıza alıyoruz.

2 Clone

Kendi hesabımızdaki (Fork) reponun linkini kopyalayıp terminalde `git clone [url]` komutu ile kodu bilgisayarımıza indiriyoruz.

3 Branch Oluştur ve Çalış

Kodu açıp kendi branch'imizi `git checkout -b yeniozellik` komutuyla oluşturuyor ve kodlarımızı yazmaya başlıyoruz.

Operasyonel Döngü II: Kodu Buluta Yollama



git add

Değişen dosyaları **sahneye alıyoruz** (staging area). Hangi değişikliklerin commit'e dahil olacağını seçiyoruz.



git commit

Sahnedeki değişiklikleri **paketleyip mühürlüyoruz**: `git commit -m "yenilikler"`



git push

Bilgisayarımızdaki güncel kayıtları, kendi GitHub hesabımızdaki (Fork) repomuza **yolluyoruz**.



Bu üç komut — `git add`, `git commit`, `git push` — günlük iş akışınızın temel döngüsüdür. Her özellik tamamlandığında bu adımları tekrarlayın.



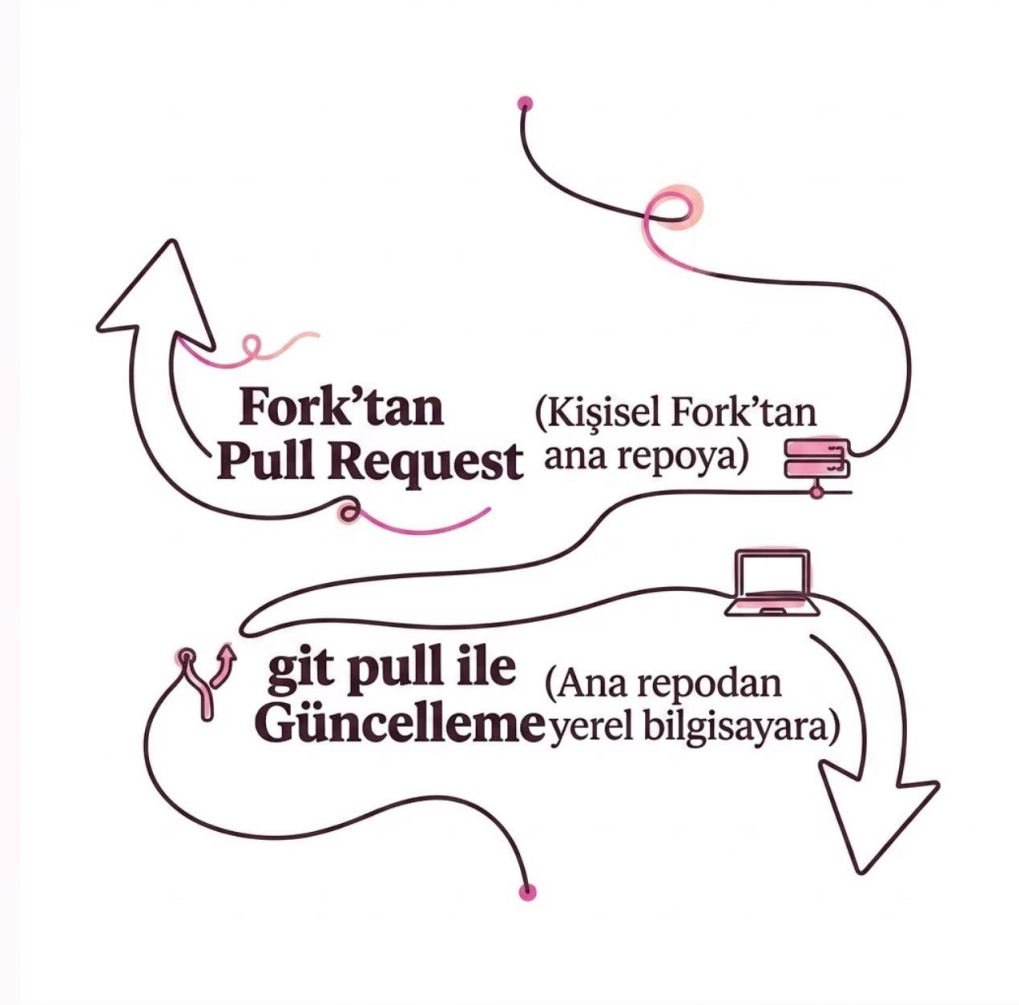
Operasyonel Döngü III: Takıma Teslim ve Senkronizasyon

Pull Request (PR) Nedir?

Kendi repomuzdaki değişiklikleri, **şirketin ana reposuna gönderme isteğidir**. "Kodlarımı yazdım, lütfen inceleyip ana projeye dahil edin" demektir. Takım lideri kodu inceler, onaylarsa birleştirir.

Sync Fork / git pull

Sen çalışırken ana repoda başkaları değişiklik yaptıysa kodun **eskir**. Bunu güncellemek için orijinal depodaki değişiklikleri bilgisayarına `git pull` ile çekersin ve projeni en güncel haline getirirsin.



● Pull Request Açma

GitHub'da "**New Pull Request**" butonuna tıkla → Değişikliklerini açıkla → Takım liderinin onayını bekle

🔄 Senkronizasyon

Ana repoda değişiklik olduğunda `git pull` ile güncelle → Çakışma varsa çöz → Çalışmaya devam et

Kriz Yönetimi: Conflict (Çakışma) Çözümü

Conflict Nedir?

Takım arkadaşınla aynı anda, **aynı dosyanın tam olarak aynı satırında** farklı kodlar yazarsanız oluşur. Git hangisini seçeceğini bilemez ve müdahale ister.

⊗ Conflict'ler kaçınılmazdır — önemli olan nasıl çözüldüğüdür. Paniklemeden, adım adım ilerleyin.

Conflict İşaretleri

```
<<<<<<< HEAD
Senin kodun buraya gelir
=====
Arkadaşının kodu buraya gelir
>>>>>>> feature-branch
```

Çözüm Adımları

01

İşaretleri Bul

Git, dosya içine <<<<<<< HEAD gibi işaretler bırakarak çakışan kısımları gösterir.

02

Manuel Karar Ver

Kod düzenleyicini açıp, kimin kodunun kalacağına (veya ikisinin birleşimine) **insan olarak manuel karar verirsin**.

03

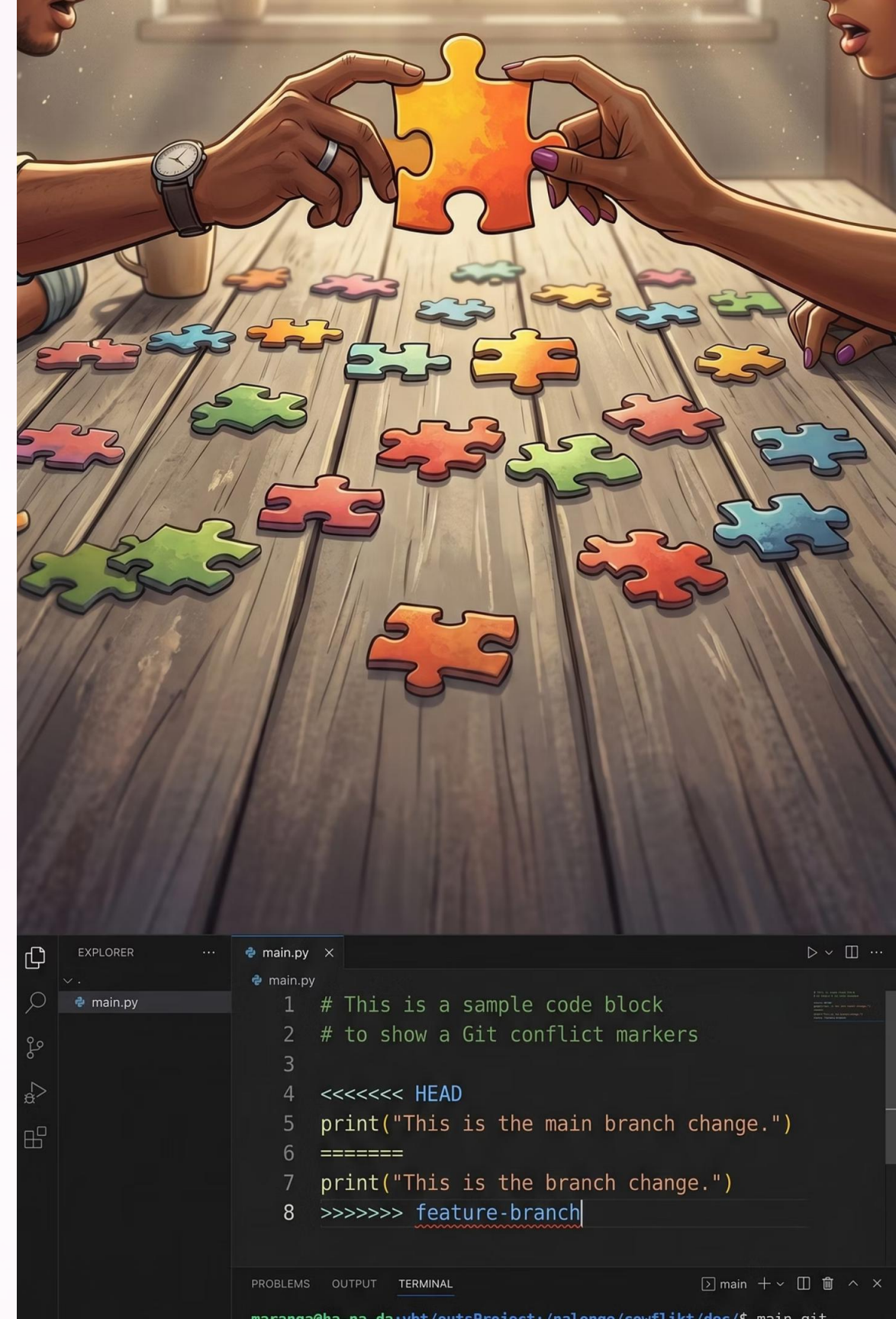
Temizle ve Kaydet

Fazlalık işaretleri sil, dosyayı kaydet.

04

Commit ile Mühürle

Yeni bir `git commit` ile çakışmayı çözüp yola devam et.



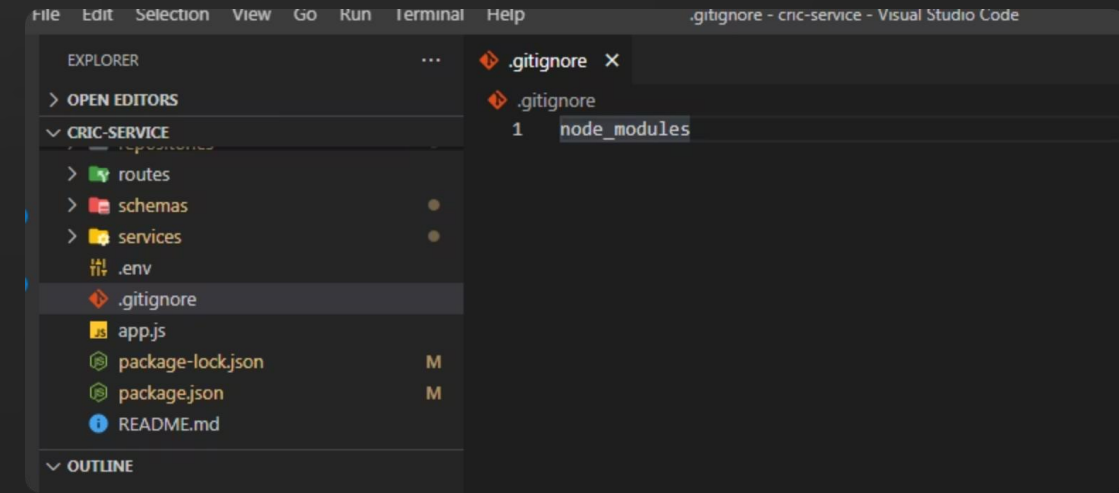


Etkili Commit Atma Sanatı

- **Nedir?:** Yaptığınız kod değişikliklerinin bir açıklama mesajıyla beraber Git tarihçesine kaydedilmesidir.
- **Kötü bir commit mesajı, taşınırken kolinin üzerine sadece "Eşyalar" yazmak gibidir; aradığınızı bulamazsınız.**
- **Nasıl Atılmalı? (Conventional Commits):** Takım içinde standart bir dil kullanmak için mesajlar **tip: açıklama** formatında yazılır.
- **Tipler ve Örnekler:**
 - 🚀 **feat:** Yeni bir özellik eklendiğinde. (Örn: **feat: kullanıcı giriş ekranı eklendi**)
 - 🐛 **fix:** Bir hata düzeltildiğinde. (Örn: **fix: şifre sıfırlama hatası giderildi**)
 - 📄 **docs:** Sadece dokümantasyonda değişiklik yapıldığında. (Örn: **docs: README dosyası güncellendi**)
 - 🧹 **chore:** Proje ayarları veya gereksiz dosyalar temizlendiğinde

Görünmez Kahraman: .gitignore

- **Nedir?:** Git'in takip etmesini ve buluta yüklemesini istemediğimiz dosyaların listelendiği özel bir ayar dosyasıdır
- **Neden Kullanırız?:** Kişisel bilgisayarımızdaki şifreleri, otomatik derlenen dosyaları, devasa paket klasörlerini (örn. `node_modules/`) veya geçici log dosyalarını projeden uzak tutmak için
- **Nasıl Yazılır? (Örnekler):**
 - `*.log` → Sonu `.log` ile biten hiçbir dosyayı takip etme
 - `node_modules/` → Bu klasörün tamamını görmezden gel
 - `.env` → Gizli şifrelerin olduğu dosyayı sakla.



Dinlediğiniz için teşekkürler .